

Documentation must include all the major sections.

You may omit specific sections not relevant to your project:

- Introduction
 - One sentence "sound bite" that describes your project.
 - A summary of what you did and why.
- High level design:
 - rationale and sources of your project idea
 - background math
 - logical structure
 - hardware/software tradeoffs
 - Discuss existing patents, copyrights, and trademarks which are relevant to your project.
- Program/hardware design:
 - program details. What parts were tricky to write?
 - hardware details. Could someone else build this based on what you have written?
 - Be sure to specifically reference any design or code you used from someone else.
 - Things you tried which did not work
- Results of the design:
 - Any and all test data, scope traces, waveforms, etc
 - speed of execution (hesitation, flicker, interactiveness, concurrency)
 - accuracy (numeric, music frequencies, video signal timing, etc)
 - how you enforced safety in the design.
 - usability by you and other people
- Conclusions:
 - Analyse your design in terms of how the results met your expectations. What might you do differently next time?
 - How did your design conform to the applicable standards?
 - Intellectual property considerations.
 - Did you reuse code or someone else's design? Did you use any of Altera's IP?
 - Did you use code in the public domain?
 - Are you reverse-engineering a design? How did you deal with patent/trademark issues?
 - Did you have to sign non-disclosure to get a sample part?
 - Are there patent opportunities for your project?

One sentence summary:

This stereo vision project utilized two cheap cameras to construct a depth map that would be displayed on a VGA screen.

Introduction

Stereo vision is a technique for extracting a 3D scene from two, 2D images. By setting up two cameras slightly offset from each other and pointed at the same picture, we can measure the difference where features appear in the two images, which is a direct function of the depth of that feature. We demonstrate stereo vision running in real time on an FPGA.

Stereo vision is a powerful technique, but it is computationally intensive. Most algorithms use block matching: for each pixel in the reference image, select a block around it, and find the block in the other image which best matches it by sliding it across the frame. This process is computationally expensive, as each pixel of the image requires many calculations on many pixels at many positions. Basic block matching is embarrassingly parallel, but realizing this parallelism requires getting pixels from memory in parallel, and reusing data read from memory optimally.

These challenges align well with the DE1-SoC. The FPGA on the board has enough configurable M10K memory for each row of pixels to be stored in its own memory bank, allowing full columns of the both frames to be accessed simultaneously. Most of the LUTs are utilized to implement the block matching algorithm, a sum of absolute differences (SAD) applied to the block. Finally, the DSPs are used for rectification, the process of transforming the warped camera image to a rectilinear projection.

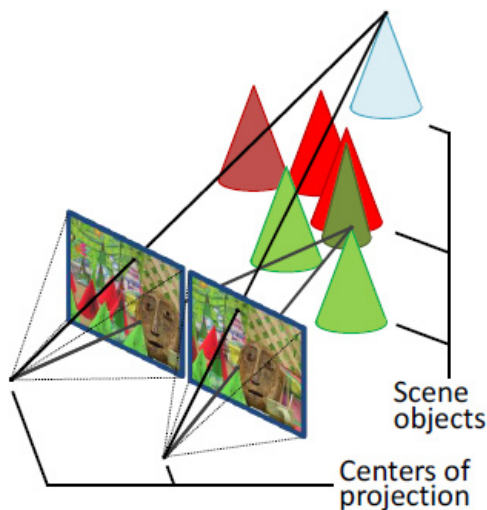
Stereo vision is an incredibly practical method of getting depth information. Cameras are cheap and typically already present in places where stereo vision may be utilized, like autonomous vehicles and robots. While physical systems like LiDAR are also capable of extracting depth information, such systems are complex, expensive, and heavy. Stereo vision can be implemented wherever there are two conventional cameras.

High Level Design

Stereo Vision

Stereo vision works by determining disparities between the same features in two frames. The disparity of a feature is defined as the difference between its position in the reference image, and the position on the other image. Disparity is inversely proportional to depth scaled by the distance between the two cameras.

We create this disparity map by block matching every pixel in the reference (left) image to a position in the other (right) image. Block matching works by selecting a rectangular section of pixels centered on the pixel whose disparity we are trying to find in the reference image. Then, starting at the same position in the other image, we compare this block to every block in the other image in one direction up to a runtime programmable maximum disparity. On each comparison, we calculate a difference between the reference and object block. When we have compared the reference block to every block up to a maximum disparity value, we pick the disparity that minimizes the difference between the reference and comparison block.



SAD

Block matching relies on a way to calculate the difference between two different blocks of pixels. We experimented with many different approaches, like a census transform and hamming distance. However, we found that we got the best results in software testing using the simplest, sum of absolute differences (SAD). SAD is simply the sum of the absolute value of

$$SAD = \sum |d_a - d_b|$$

each pixel difference:
of the difference.

. This is in essence an L1 norm

1-Directional Semi-Global Matching

Semi-Global matching is an algorithm used in computer vision to compute a dense disparity map using a pair of rectified stereo images. This algorithm works by computing the similarity of each pixel on one stereo image with each pixel in a subset of pixels from the other stereo image. The math works as follows: for each pixel with coordinates (x, y) , the set of pixels from the other stereo image is selected as $\{(x(\text{hat}), y) | x(\text{hat}) \geq x, x(\text{hat}) \leq x + D\}$, where D is selected as the maximum allowed disparity shift.

However, relying only on a pixel matching algorithm would produce a lot of incorrect matches especially in textureless or monocolored areas. To combat this the algorithm uses a cost function that applies penalties for having vastly different disparities for adjacent/close pixels. Generally the cost function to be minimized is of the form (from wikipedia, remake ourselves on da website)

$$E(\mathbf{d}) = \sum_p D(p, d_p) + \sum_{p,q \in \mathcal{N}} R(p, d_p, q, d_q)$$

Where $D(p, d_p)$ is the dissimilarity cost that represents how different the pixels look from each other, and $R(p, d_p, q, d_q)$ is the smoothness cost that represents how different the disparity map pixels are in a given region. The semi-global matching algorithm attempts to minimize this equation by finding the point that is most similar to the initial coordinate, while making sure that the final disparity map pixel remains smooth with respect to its surrounding pixels.

In this project we adapted the algorithm to better suit our needs. Standard SGM reduces the 2D optimization problem above into a sum of 1D optimizations along multiple scan paths. For each path direction the algorithm recursively adds a cost along that direction. Full multidirectional aggregation is great at smoothing but a poor choice for our project. Each path direction would need its own running cost array of size D in memory, and the reverse direction path would require its own copy of the cost volumes or a second pass over the frame. This was especially bad for us because our SRAM was already saturated. We instead collapsed the smoothness down to a single path (1D) along the horizontal left to right direction, which is the natural sweeping order of the block matcher. Each pixel p with a candidate disparity of d_p the penalty cost to be minimized was $C(p, d_p) = \text{SAD}(p, d_p) + R(d_p, d_{p-1})$

Lense Rectification

Software Pathfinding

The rectification side of the OpenCV work walked from "no calibration, just an analytic fisheye model" to "real calibration baked into the FPGA as a small polynomial." First pass was [python-testing/fisheye-reconstruction.py](#), which parameterises the two candidate Razer lenses (125° and 98° HFOV) under different analytic projections – equidistant, equisolid, stereographic, rectilinear – and remaps the cropped 320×240 active region with `cv2.remap`, blending between identity and the model by a `correction_strength` knob; the autotune family `v6..v20` ([autotune_v6.py](#), [classify_lens_from_scratch_v16.py](#), and friends) then scored each candidate remap with `cv2.Sobel` edge tracking and `cv2.Canny` + `cv2.HoughLinesP` "is this line straight after warp?" objectives. This never produced a usable stereo pair – the center looked corrected, the periphery didn't – so we switched to a real calibration in [python-testing/camera_calibration_charuco.py](#) and [python-testing/split_calibrate_generate_luts_and_compare.py](#): CLAHE + 3× upscale, `cv2.findChessboardCornersSB` (fallback `findChessboardCorners` + `cornerSubPix`), `cv2.fisheye.calibrate` for each lens independently, then `estimateNewCameraMatrixForUndistortRectify` → `initUndistortRectifyMap` → `remap`. The resulting per-pixel gather map was packed by [generate_stereo_lut_from_existing_calibration.py](#) into the 21-bit {valid, src_y, src_x} ROM format the RTL consumes, and then [fit_polynomial_mapper_from_lut.py](#) plus [radial_shift_degree_sweep.py](#) compressed the LUT into a small fixed-point polynomial – a 2D deg-6 fit in Q18 (now [stereo_poly_mapper_deg6_q18.v](#)) and a purely radial fit in Q15 with an optional center shift (now [radial_scale_lut_q15.vh](#) consumed by `stereo_radial_mapper_q15*.v`). The polynomial form is what survives in the RTL because the full LUT is too big to keep on-chip alongside the SAD engine and framebuffers.

The first family of disparity experiments lived in [sterio-vision-python/stereo_depth_system/methods.py](#) and was about finding the right **matcher and operating point**. We wrapped OpenCV's own `cv2.StereoBM_create` and `cv2.StereoSGBM_create` (with `P1 = 8·blockSize²`, `P2 = 32·blockSize²`, `STEREO_SGBM_MODE_SGBM_3WAY`, `displ2MaxDiff=1`, `uniquenessRatio=10`, `speckleWindowSize=50`) as a golden reference, then wrote a custom windowed-SAD from scratch using `cv2.boxFilter` for the per-disparity aggregation step and a winner-take-all loop over `d`. Both an L1 and an L2 cost mode are in there (`compute_sad_l1`, `compute_sad_l2`), and the sweep driver in [run_stereo_experiments.py](#) walked `--sad-max-disparities "24,32,48,64,96"` against `--sad-window-sizes "5,7,9,11,13"` while a parallel script [disparity_sweep_pair_003_004.py](#) pushed the window/disparity grid much further (window 3..33, max-disp up to 224) to see how flat the metric landscape was. The result of those sweeps is what's now hardwired into [block_match_pkg.sv](#) and [stereo_config.h](#): 9×9 window, max-disp 32. That number isn't arbitrary; it's the smallest setting in the sweep where the disparity map kept its structure on our specific 320×288 captures. The SGBM penalty constants came along the same way – `SGM_P1/SGM_P2` exposed as parameters on [block_match.sv](#) (commits "SAD penalties", "configurable penalties") are the same `P1 = 8·blockSize² / P2 = 32·blockSize²` formula OpenCV uses internally.

The second family was about **changing the cost metric and the input preprocessing**, because raw absolute-difference SAD struggled on low-texture and exposure-mismatched regions. We added a Census + Hamming path (`compute_census_hamming_custom`) that replaces each pixel with the sign-pattern of its neighbours in a `census_window` and then matches by `cv2.boxFilter` of popcount-XOR distances – invariant to local brightness differences between the two cameras, which we definitely had. The sweep driver also runs a preprocessing grid over CLAHE on/off and median-blur kernel size 0/3/5

(`--preprocess-sweep`), and [disparity_sweep_pair_003_004.py](#) tries a Gaussian high-pass (`cv2.GaussianBlur` then subtract) before matching. The conclusion was mixed: Census beat SAD on the textureless surfaces but was worse on edges with real depth steps, and the preprocessing knobs each fixed one failure mode while introducing another. So the RTL keeps SAD as the primary path and leaves Census as a viable parallel implementation we can swap in, with CLAHE and median-blur retained as preprocessing knobs on the C reference rather than burned into the RTL.

The third family was the **2D (d, vy) search** — and this one is the cleanest example of a Python finding that we could not get into the FPGA in time. OpenCV `cv2.StereoSGBM` assumes perfectly horizontal epipolar lines, and ours weren't: each lens was calibrated independently, the two cameras weren't perfectly co-planar, and remapping left sub-pixel row error that SGBM could not see through. So `compute_sad_custom_vshift` and `compute_sgbm_vshift` in [methods.py](#) extend the search to also try $\pm v$ row shifts of the right image for each candidate d and keep the (d, vy) pair with minimum aggregated cost — the `vshift_color.png` preview (blue=shift up, green=zero, red=shift down) shows that real images need 1–3 rows of slack across the frame. This was the largest-quality jump in the whole Python sweep (commit [6b24f57a](#) literally says "significantly improving the disparity map produced"), and it's a first-class parameter in the C reference matcher (`v_shift / --vshift`). On the RTL side we never shipped it — getting the row-banked on-chip RAM, the per-stripe SAD units, and the radial-mapper ring buffer all routed and meeting timing ate the project's time budget, and a 2D (d, vy) search would have needed the SAD engine to consume `BLOCK_SIZE + 2 · V_SHIFT` rows of right-image context at once instead of `BLOCK_SIZE`. So `vshift` stayed a Python/C-only result that explains the residual row-misalignment artifacts we still see in the FPGA disparity map.

Hardware Design

Our stereo vision system is built on the Terasic DE1-SoC. The Cyclone V FPGA fabric hosts a baseline system that provides a bridge to the ARM HPS, a Video-In subsystem that decodes the analog camera feeds into a single interlaced frame, and a VGA-out pixel DMA driving a framebuffer in SDRAM. To achieve real-time performance, we extended this baseline with three custom hardware subsystems that together form our depth pipeline:

1. **Row-Banked On-Chip Memory:** A custom memory architecture that exposes a parallel read bus, allowing the SAD engine to fetch one byte from every row of the frame simultaneously.
2. **Radial Undistortion Pass:** An in-place rectification pipeline that corrects lens distortion before matching begins.
3. **Parallel Block-Matching Engine:** A 32-way parallel SAD compute unit that produces a dense disparity map, incorporating an SGM-inspired smoothness penalty.

These subsystems are sequenced by a four-state top-level finite state machine (FSM) that cycles through filling memory with rectified pixels, draining staging buffers, running the SAD compute, and reading back the results. Crucial parameters like window size, maximum disparity, and stripe height are configured at synthesis to match our software models, while SGM penalties are exposed to the HPS for runtime tuning.

Row-Banked On-Chip Memory

A key constraint shaped our entire architecture: a 9x9 reference block compared against a wider search window requires fetching dozens of pixels per disparity hypothesis. A standard single-port memory caps throughput at one pixel per cycle, which would make the cost-per-disparity dozens of clock cycles.

To realize the embarrassingly parallel nature of SAD, we built a custom memory architecture that physically banks the frame along its Y-axis. Each of the 200 image rows is stored in its own dedicated memory block. While write access is multiplexed between the Video-In DMA and the HPS, we implemented a private, parallel read bus. This allows the hardware to fetch an entire column of pixels—one byte from every row—in a single clock cycle. This massive increase in read bandwidth is what makes the 32-way parallel SAD economically reachable.

Implementing this massive fan-in required specific architectural optimizations to allow the design to successfully route and close timing:

- **Direct Memory Instantiation:** Relying on standard inference for the memory blocks proved unreliable across different coding styles, leading to internal synthesis errors. We explicitly instantiated dual-port RAM primitives for each row to ensure stability.
- **Two-Stage Read-Data Multiplexing:** The 200-way multiplexer required for the read bus created a severe routing hotspot. We resolved this by splitting it into a registered 16-way

intra-group multiplexer followed by a 13-way inter-group multiplexer. This added a cycle of latency but dropped placement and routing complexity by an order of magnitude.

- **Global Write Arbiter:** Rather than replicating write logic 200 times, a single arbiter broadcasts the winning write data and address to every row, with per-row logic gating only the byte enables. This saved over 3,000 ALMs of logic resources.

In-Place Radial Undistortion

Before block matching can occur, lens distortion is corrected directly on the FPGA to ensure the hardware engine processes the exact same rectilinear projection as our software models. The radial mapper uses an 11-stage fixed-point pipeline. It consumes one destination pixel per cycle and emits the corresponding source coordinate eleven cycles later—calculating the vector to the image center, applying a radial coefficient, interpolating from a calibration lookup table, and applying center offsets.

Because on-chip memory is highly constrained, this undistortion pass must rewrite the SRAM in place. However, undistortion naturally shuffles pixels vertically; writing a rectified pixel directly back into memory could overwrite source pixels that are still needed for subsequent operations. To solve this, we implemented a staging ring consisting of multiple row buffers. This ring holds the newly rectified pixels in flight just long enough for the write-back to be safely delayed past any dependent reads.

To prevent the autonomous, free-running video DMA from overwriting freshly-rectified bytes during compute windows, the memory includes a dedicated hardware lock. When asserted, incoming video writes are silently dropped, keeping the data stable for the SAD engine.

Radial Mapper FSM

The mapper is an eleven-state machine that takes one destination pixel (dst_x, dst_y) and produces one source pixel (src_x, src_y) over eleven clock cycles. In state one (`ST_IDLE`) it latches the destination coordinate, decides whether the pixel lives in the left or right half of the side-by-side frame, and loads the four per-side calibration constants — the destination lens center (C_x, C_y) and the source lens center $(C_{\{src,x\}}, C_{\{src,y\}})$. In state two (`ST_VEC`) it subtracts the destination center to get a Q8 offset vector (dx, dy) pointing from the lens center to the pixel. In state three (`ST_R2`) it squares each component and sums them to get r^2 , the squared radial distance from the lens center. In state four (`ST_POS`) it multiplies r^2 by the constant 92 to produce a 32-bit Q16 LUT position, whose upper bits are the LUT index and whose lower sixteen bits are the interpolation fraction between adjacent table entries. In state five (`ST_LUT`) it reads the two LUT entries flanking that index, $s_0 = LUT[idx]$ and $s_1 = LUT[idx+1]$, clamping to the last entry past the end of the table. In state six (`ST_INTERP`) it computes the difference $(s_1 - s_0)$ and multiplies it by the saved fraction, giving the partial slope-times-distance term of a linear interpolation. In state seven (`ST_SCALE`) it adds that interpolated correction back onto s_0 (with round-to-nearest) to finish the interpolation and produce a smooth Q15 `scale` factor for this exact radius. In state eight (`ST_MUL`) it multiplies the original (dx, dy) vector by that `scale`, stretching or shrinking it radially to the source-side geometry. In state nine (`ST_OFFSET`) it re-anchors the scaled vector on the source-side lens center by adding $(C_{\{src,x\}}, C_{\{src,y\}})$, producing the candidate source

coordinate in Q8 fixed-point. In state ten (`ST_CHECK`) it rounds the Q8 coordinate to integer pixels, applies the right-half offset of 319 if appropriate, bounds-checks against the captured-frame size, and drives the output registers `src_x`, `src_y`, and `valid`. Finally, in state eleven (`ST_DONE`) it pulses `done`, drops `busy`, and returns to `ST_IDLE`, ready to accept the next destination pixel — and because this is a strictly sequential state machine rather than a pipelined one, that next pixel has to wait until the current eleven-cycle traversal completes before it can begin.

Radial Distortion Scheduler

The whole pipeline is shaped by two hard constraints. First, the on-chip SRAM is only about 200 KB, which is just barely enough to hold one 640×288 grayscale frame. There is not enough room for both a raw copy and a rectified copy, so the rectified frame has to overwrite the raw frame in the same memory, byte for byte. Second, the Video-In subsystem's DMA is autonomous: it streams freshly captured PAL frames into the SRAM continuously on its own clock, without coordinating with anything else on the chip. Every odd piece of the rectification pipeline is a direct consequence of one of these two facts.

Rectification itself is conceptually simple. For every output pixel (`dst_x`, `dst_y`) we want to produce, the radial mapper tells us which raw pixel (`src_x`, `src_y`) it came from in the captured frame, so we read that raw byte out of SRAM and stash a rectified byte back into SRAM at the destination position. The catch is that the lens distortion makes the mapper's source row drift by up to about twelve rows above or below the destination row. So while we are working on rectified row 60, the mapper might still ask us to read raw row 50 as a source pixel. If we had already overwritten raw row 50 with its rectified version several rows earlier, we would now be reading our own rectified output as input, which produces garbage. The general rule is that raw row `r` is safe to overwrite only once every later destination row that might still touch it is done reading from it. With a maximum vertical drift of `K = 12`, that means raw row `r` is safe only after destination row `r + K` is finished.

This is where the `undistort_ring` comes in. Instead of writing each rectified row straight back into SRAM, we stash it in a thirteen-slot circular buffer for `K` more destination rows before committing it. When we finish destination row 0 we put it in slot 0 of the ring; when we finish row 1 we put it in slot 1; and so on until row 12 fills slot 12. Once we finish row 13 the ring is full, so before we put row 13 into slot 0 we first drain the old contents of slot 0 (rectified row 0) into SRAM row 0. At that moment we are past row 13, the worst-case source row we could still need is row 0, and we are already done with it — so raw row 0 is safe to overwrite. Every subsequent destination row does the same thing: the ring is a `K`-row delay line that guarantees the safety margin holds for every raw row. When `PHASE_FILL` ends after destination row 287, the last thirteen rectified rows are still sitting in the ring waiting their turn to be committed. `PHASE_DRAIN` exists for exactly that reason — it walks the remaining slots and flushes each one to its matching SRAM row, so by the time it finishes the SRAM holds a fully rectified frame.

Now the second constraint comes back to bite us. The Video-In DMA, left to itself, would just keep writing fresh raw frames into the SRAM all the way through `PHASE_FILL`, `PHASE_DRAIN`, and `PHASE_COMPUTE`, randomly clobbering rectified rows that we just produced and feeding the SAD matcher a mix of rectified and raw bytes. To stop that from happening, the design holds a write-lock on the SRAM's `s1` port for the entire `FILL`-through-`COMPUTE` window, which makes the DMA's writes silently fail. That lock is essential for SAD to see a consistent frame. But it also means that while the lock is held no new frames are getting captured, so if we never released the lock we would just rectify and match the same picture forever.

`PHASE_WAIT` solves that: it releases the lock for about 50 ms, which is comfortably longer than one 40-ms PAL frame, so the DMA is guaranteed to lay down at least one complete fresh capture during that window. Once `WAIT` ends the lock re-asserts and the next `FILL` begins.

Put together, the system runs an endless loop of four phases. WAIT lets the DMA refresh the SRAM with a new raw frame. FILL walks every destination pixel in raster order, the mapper produces source coordinates, raw bytes come out of SRAM, rectified bytes go into the ring, and the ring's oldest slot continuously drains back into SRAM in place. DRAIN flushes the last thirteen rectified rows still left in the ring after FILL ends. COMPUTE runs the SAD matcher across the now fully rectified SRAM via port B of every M10K in parallel. Then the cycle returns to WAIT. The ring buffer is just the delay you need to make in-place overwriting safe given the mapper's row-shift range; the WAIT phase is just the breathing room you need to give the DMA so it can refresh the frame in spite of the write-lock you have to hold the rest of the time. Both are forced moves once you commit to fitting only one frame on chip and sharing that frame with an autonomous capture path.

The Four-Phase Top-Level Scheduler

To safely share memory without read/write conflicts, the top-level FSM serializes the subsystems into four distinct phases:

- **Fill Phase:** The radial mapper processes destination pixels, writing the rectified source bytes into the staging ring. Concurrently, a minor FSM drains the oldest ring slots back into the main SRAM.
- **Drain Phase:** Once the frame is processed, this brief tail phase allows the minor FSM to flush the remainder of the staging ring back into SRAM.
- **Compute Phase:** The SRAM is locked to prevent video overwrites, and the parallel SAD engine is triggered. It streams calculated disparities directly to the VGA framebuffer, allowing the user to view the disparity map in real time.
- **Readback:** Once the engine signals completion, the FSM implicitly loops back to the Fill Phase for the next frame.

The Parallel SAD Block-Matching Engine

The block matcher is the core of the compute pipeline, implementing a parallel-stripe sweep over the rectified frames through several interconnected components:

- **Striping:** The 200-row frame is divided into 32 horizontal stripes, each 6 rows high. Each stripe is assigned its own dedicated SAD engine with a private sliding window. All 32 engines are clocked together, evaluating one disparity hypothesis at 32 different vertical positions simultaneously.
- **Column Prefetch:** A three-state handshake interface sits between the main engine and the SRAM. It translates stripe requests into parallel memory reads, latching full columns for both the reference and search images.
- **Sliding Window:** To optimally reuse data read from memory, each stripe utilizes a shift register file for its reference block. As the window shifts right across the image, it only needs to insert the freshly-prefetched column, keeping the rest of the block stable and eliminating redundant reads.

- **Outer FSM:** The control logic manages three nested loops: sweeping the disparity hypothesis up to the maximum limit, sliding the reference block along the X-axis, and shifting the windows down the Y-axis to cover the full stripe.
- **1-Directional Semi-Global Matching (SGM):** We integrated our adapted SGM algorithm directly into the hardware. For each disparity hypothesis, the engine calculates a penalized cost: the raw SAD value plus a smoothness penalty. If the hypothesis differs from the adjacent pixel's best disparity by one, a small penalty is applied; if it differs by more, a larger penalty is applied. This single-pass approximation of SGM requires minimal hardware (a small adder and multiplexer per unit) but significantly smooths the output.

HPS Software Design

The HPS software does not perform any load-bearing computational tasks. It is used as the user interface, allowing the user to set parameters from the command line. The command line interface is implemented as a simple read-eval-print loop (REPL). The user can set a parameter by typing the name of the parameter and value, e.g. “maxd 85” sets the maximum disparity value to 85. Updates to parameter values are processed in real time. The HPS maps a PIO port for each of four configurable parameters:

- **Maxd:** the maximum disparity value which is checked.
- **Mind:** the minimum disparity value which is checked.
- **P1:** The penalty assigned to a disparity value one away from the previous value.
- **P2:** The penalty assigned to a disparity value more than one away from the previous value.
- **Edge:** Enable the Altera Edge Detection IP

The HPS also sets default values, so that the FPGA can correctly pick up parameter values. How the HPS reaches the FPGA

Software Control Interface

Communication between the ARM processor and the FPGA is handled through Parallel I/O (PIO) ports via the Lightweight HPS-to-FPGA bridge. The C program running on the ARM processor maps the physical hardware addresses—including the bridge, VGA buffers, and SRAM—into virtual memory using Linux's /dev/mem interface.

Because the FPGA-side PIOs are strictly output-only from the processor's perspective, the software maintains a local mirror of all tuning parameters. Upon booting, the program immediately initializes the hardware with a known-good default configuration, establishing baseline SGM penalties and disparity bounds so the stereo pipeline begins functioning immediately.

The core of the software operates as a lightweight read-eval-print loop (REPL). It accepts terminal commands over SSH, allowing the user to adjust hardware parameters on the fly. Through this interface, users can dynamically tune the semi-global matching penalties, alter maximum and minimum disparity bounds, or toggle edge detection. The software sanitizes user input—such as clamping negative values—and pushes the updates to the hardware registers, altering the FPGA's behavior on the very next frame.

Frame Capture Utility

To evaluate the hardware's performance, we integrated a custom frame-capture utility directly into the control software. When triggered via the REPL, the software reads the SDRAM-backed VGA framebuffer and constructs a complete, uncompressed PNG file entirely from scratch. We created a lightweight tool to capture and save disparity maps directly from the board. We then copy these of the HPS using the `scp` command.

Results

Speed

We were able to synthesize our design with 24 parallel SAD units executing simultaneously. At this level of parallel computation, we found that the framerate was ultimately bottlenecked by the undistortion pipeline. When the pipeline is full, it drops frames from the video input subsystem. In practice, this results in a 50 ms pause between a new frame, ultimately limiting the framerate with rectification enabled to 20 FPS.

With rectification disabled, we got worse results, since rectification is necessary for horizontal sweeps through the frame to correspond exactly with disparity changes. However, we did find a significant improvement in performance. With $N = 24$, the frame was split into $\text{ceil}(288/24)$ tall stripes, and each worker had to do $\text{stripe_height} * \text{frame_width} * \text{max_disp} + \text{frame_width} * 5 + \text{stripe_height} * 5$ computations per frame. For our setup, this was approximately 231,672 cycles. At the achieved clock frequency of 50 MHz, this meant that a single frame was computed in 0.00463 ms, corresponding with a frame rate of 215 FPS, significantly exceeding the 25 FPS framerate of the PAL video standard we were using. In retrospect, dedicating more hardware resources and design effort to speeding up the rectification pipeline would have made computing faster than the 25 FPS PAL cameras achievable. However, this also shows that our highly parallel block matching implementation is incredibly performant, and would scale well to more frequent rectified video input.

Accuracy

We found that the stereo vision system performed the best at a distance of 1-3 m from the camera setup, with a camera distance of 6 cm. At this distance, it is possible to differentiate objects in the foreground from the background, with bodies and objects appearing as light, monochrome silhouettes against a dark, noisy background. The vision system is additionally able to pick up on gradients for objects which are angled towards / away from the camera, showing the change in the object's depth from the screen.

A known issue with basic block matching is that it performs poorly on monochromatic, untextured surfaces. This is because block matching assigns similar costs to every disparity within the surface, because they all appear identical without a priori knowledge of where that part of the block is within the surface. We ameliorate this using a simple, SGM inspired penalty to SAD values to smooth out surfaces. This 1-way smoothing creates a noticeable difference between the left and right edges of objects, but it does significantly increase performance on monochrome surfaces, and larger objects in general, because it penalizes discontinuities in the disparity map.

Resource Utilization

We were able to successfully compile our design with a block size of 5 and 24 parallel SAD workers. We found that the limiting resource in the size of the design that we could

synthesize was the ALM usage needed to create larger or greater numbers of SAD compute units. Increasing the number of workers increases the ALM count roughly linearly, but increasing the block size increases the number of workers exponentially with respect to the radius of the block. A block size of 5x5 was the largest we could achieve while still having enough workers to not bottleneck the disparity map computation.

We initially struggled with M10K memory block usage because we inadvertently double-buffered incoming frame data. This was an artifact of two separate architectural decisions: the first was to put incoming pixel data into memory that was fragmented row-wise, the second was to store a rectified frame in that memory. As a result, we were initially using two separate memories for frame data: one was a monolithic on-chip SRAM synthesized from many M10Ks, which was written to by the video-in subsystem. Data from this memory was then rectified and copied to the segmented memory where it would be read in parallel by the SAD compute units. To resolve this redundancy, we wrote our own memory block, which exposed two interfaces: an Avalon MMIO interface to be written to by video input, and a parallel, asynchronous interface to be read from by the compute unit. This effectively halved our M10K utilization. Importantly, it allowed us to use a separate M10K unit for each row of each image, which allowed for maximum data parallelism. This memory block required many (13268) ALMs, as the MMIO interface required muxing between 200 separate block rams which are physically spread across the chip. This also led to very long compile times, as the fitter would struggle with this routing issue.

Conclusion

Our project demonstrated that fast block matching and stereo vision is achievable on the DE1-SoC using inexpensive PAL cameras. We were successfully able to demonstrate rectification, highly performant and parallel block matching, and a realistic depth map drawn to the VGA screen. Achieving this goal required solving a variety of problems. We had to measure and understand the geometry of the camera lenses we were using, utilizing OpenCV to calibrate our cameras. We tested a number of different block matching algorithms with different parameters and preprocessing techniques against our real-life camera setup, allowing us to thoroughly search the design space before implementing anything on the FPGA. It also required us to thoroughly understand the Cyclone 5's capabilities, in particular the configuration of on chip memory and utilization of the Avalon MMIO and streaming interfaces, enabling our blocks to talk directly to the Avalon video input IP. Finally, it required thorough consideration of the hardware architecture that was best suited for the task.

There are some architectural decisions that we made that either overcomplicated or constrained our design from the get-go that we would change for a future implementation. For starters, our implementation relies on storing an entire frame of video in memory, and then exploiting row parallelism to speed up computation. Because lines of video are received sequentially, a better approach would be to keep a much smaller line buffer of horizontal lines of video input, and parallelized across disparity sweeps rather than rows. This would have allowed us the same or better performance, as we would be able to exploit the same level of parallelism while using much less memory and ALMs.

Additionally, the greatest bottleneck on our video throughput was the rectification process. The latency for the un-pipelined rectification was 90 ms, significantly longer than the 4 ms it took to compute the disparity map. Initially, we did not realize that this would even be necessary, and so we did not lay out a clear architecture for how the process should work. The clearest way to do the rectification would be to simply modify pixel coordinates in stream with a look-up table. We encountered two issues: one was that it was difficult to reverse the mapping from rectified coordinates to actual coordinates, and the second was memory usage for the look up table. This combination of challenges led us to a massively overcomplicated multi-cycle rectification process that consumed many cycles of latency. This overcomplicated what should have been a relatively simple transformation that in principle could have been done serially in stream with a minimal impact on frame throughput. If we had anticipated that this would dominate the computation time, we would have spent much more design effort and been willing to spend more on-chip resources to simplify the pipeline.

We utilized some Altera IP in our design, in particular the video input IP, DMA controllers, VGA driver, and the memory mapped IO standard. We used OpenCV for offline lens calibration and design space exploration. We used iVerilog for RTL simulation and testing. We did not use any other code in our RTL or HPS code.

Work Distribution and AI

We divided work for this project equally across members. Ezra contributed most significantly to the offline (software) design space exploration, as well as the lens rectification and memory banking RTL. Demetrios contributed most significantly to the block matching RTL and top-level architecture. Daniel helped with RTL simulation.

We utilized generative AI tools to varying degrees for different parts of the project. For the block matching RTL, we wrote the majority of the RTL ourselves, and used generative AI tools (Claude Code, Microsoft Copilot) to identify bugs, or fix bugs that we had identified. This was useful for catching subtle bugs (e.g. we mistakenly wrote a shift register that did not shift the last index) and simplified the process of fixing more trivial bugs (e.g. small changes to state transition logic that we noticed in the waveform). For the segmented memory with Avalon MMIO integration, we used generative AI to tweak our RTL to conform to the Avalon interface standard, as well as to write a TCL script to get our module to appear in the Qsys GUI as a piece of IP that could be instantiated. The lens rectification RTL was more significantly AI driven. On the software side, it generated most of the OpenCV code to extract lens parameters from a set of images, and on the hardware side we used it to generate a number of different implementations targeting different hardware utilization. We also used generative AI heavily for software design-space exploration: it allowed us to easily try out different block matching algorithms combined with different preprocessing steps and filtering. Finally, we used AI to generate some figures and interactive animations for this report, by giving it snippets of our RTL and asking it to create different kinds of displays (e.g., the interactive block matching animation).

We found that generative AI was in some ways a productivity multiplier, but in other ways made bad ideas tractable. AI was most effective for design space exploration, because it allowed us to get a sense of what kinds of results were achievable and how very quickly. It also accelerated the process of writing RTL by catching and fixing bugs and writing useful testbenches. However in some places it made overly complicated and poorly architected solutions that we wouldn't attempt to write on our own possible. In particular, the RTL for the rectification pipeline, which was significantly AI generated, was needlessly complex and served as a major performance bottleneck. Although the solution was correct, it was incredibly difficult to optimize and modify. On the other hand, our block matching RTL was almost entirely written by us, just utilizing AI to test and fix bugs. This RTL was comparatively very performant, easy to iterate on, and easy to understand conceptually. While the ability of generative AI to produce correct RTL is impressive, it tends to create solutions that are complex and difficult to iterate on without rigid guidance.